

Capstone Project Diabetics Prediction Using Machine Learning

Presented By

Student Name: KrishnaMohan B

College Name: AmityUniversityOnline

Department: Computer Applications

Email ID:krishnamohanb21@gmail.com

AICTE Student ID: STU67c74640ed4b11741112896

Contents

- 1) Problem Statement
- 2) Proposed System
- 3) System Approach
- 4) Results
- 5) Algorithm and Deployment
- 6) Conclusion
- 7) Future scope
- 8) References

Problem Statement of Diabetics Prediction Using Machine Learning

Diabetes is a serious medical condition found in millions across the globe. Early diagnosis will help control and treat the illness effectively, alleviating complications as well as helping improve patient prognosis. The target of this machine learning project is to create an accurate predictive model that can well determine if one is diabetic through a given collection of clinical as well as demographic attributes.

There are patient records with the given features:

Demographic characteristics: Gender, Age

Symptoms and clinical indicators: Polyuria, Polydipsia, Sudden Weight Loss, Weakness, Polyphagia, Genital Thrush, Visual Blurring, Itching, Irritability, Delayed Healing, Partial Paresis, Muscle Stiffness, Alopecia, Obesity

Target variable: Class (diabetic or non-diabetic)

Proposed System of Diabetics Prediction Using Machine Learning

The system will predict whether a patient will have diabetes or not based on the features mentioned in the dataset, including Age, Gender, Polyuria, Polydipsia, etc. The solution will include collecting data, preprocessing it, training machine learning algorithms, deploying the model, and measuring its performance.

1. Data Collection

Historical Data: The data would be patient records with details of their age, gender, symptoms (e.g., polyuria, polydipsia), and other clinical characteristics, including the class label (diabetic or non-diabetic).

example dataset attributes:

Age: Patient's age. **Gender:** Patient's gender. **Polyuria:** Frequent urination (binary: Yes/No). **Polydipsia:** Excessive thirst (binary: Yes/No).

Sudden weight loss: Loss of weight without effort (binary: Yes/No). **Weakness:** Lack of energy or fatigue (binary: Yes/No).

Polyphagia: Increased hunger (binary: Yes/No). **Genital thrush:** Symptoms of fungal infection (binary: Yes/No).

Visual blurring: Blurry vision (binary: Yes/No).

Itching: Skin irritation (binary: Yes/No). **Irritability:** Feeling of easy annoyance (binary: Yes/No). **Delayed healing:**

Slowness of wound healing (binary: Yes/No). **Partial paresis:** Weakness of muscles or partial paralysis (binary:

Yes/No). **Stiffness in muscles:** Rigidity or inflexibility in muscles (binary: Yes/No). **Alopecia:** Baldness (binary:

Yes/No). **Obesity:** Overweight condition (binary: Yes/No). **Class:** Target variable, 1 for diabetic and 0 for nondiabetic.

Real-time Data: Depending on availability, other features such as patient lifestyle, diet, and exercise may be added to enhance model precision.

2. Data Preprocessing

The data must be cleaned and preprocessed prior to feeding it into the machine learning algorithms:

Preprocessing steps:

Handle Missing Values: Detect and fill missing values or drop rows with considerable missing data.

Convert Categorical Features to Numerical: Features such as Polyuria, Polydipsia, Gender, and others may need to be encoded (e.g., Yes = 1, No = 0).

Feature Scaling: Scale continuous features such as Age and Obesity via normalization or standardization (e.g., Min-Max scaling or Standard Scaler).

Split the Data: Split the data into training and testing sets, typically an 80/20 split (80% training, 20% testing).

3. Machine Learning Algorithm

There are various machine learning models that can be employed to predict if an individual has diabetes from the given features:

Logistic Regression: An easy-to-use and efficient algorithm for binary classification.

Decision Trees: Can facilitate visualization of the decision-making process.

Random Forest: An ensemble technique that tends to improve over decision trees.

Support Vector Machine (SVM): A strong algorithm for binary classification problems.

K-Nearest Neighbors (KNN): A non-parametric algorithm that classifies by proximity to closest data points

4. Deployment

After you have trained a model that works well, you can deploy it in a real-world application.

User Interface: Design a web interface where users can enter their details (e.g., Age, Gender, Polyuria, Polydipsia sudden weight loss weakness Polyphagia Genital thrush visual blurring Itching Irritability delayed healing partial paresis muscle stiffness Alopecia Obesity etc.) to predict if they have diabetes.

Use Flask for designing the web interface.

The app will accept the user input, feed it into the model, and output a prediction (e.g., diabetes risk = "Yes" or "No").

5. Evaluation

Once deployed, continuously evaluate the performance of the model. Use metrics such as:

Accuracy: It measures the ratio of correct predictions.

Precision: It measures how many of the predicted positive cases (diabetic) are indeed positive.

Recall: It measures how many actual positive cases were picked up by the model.

F1 Score: Harmonic mean of precision and recall.

6. Result

The last output is a system that predicts whether an individual is likely to have diabetes given their clinical features. The deployed system enables the user to feed in their details and receive real-time prediction for their risk of diabetes.

System Approach

System Requirements

HARDWARE REQUIREMENTS : The hardware requirements for running this website and model are:

RAM – 8.00 GB

Operating System – Windows 11

Processor – Intel(R) Core(TM) i3-1115g4

Processor speed – 3.00 GHz

SOFTWARE REQUIREMENTS: The programming language used to develop this application is Python and the IDE used is

Jupyter Notebook. Front end is made using HTML and is integrated with flask.

Programming Language – Python

Python IDE – Jupyter Notebook

Python Libraries: Flask

Libraries to build the model

- 1) Pandas
- 2) matplotlib
- 3) seaborn
- 4) sklearn
- 5) Flask

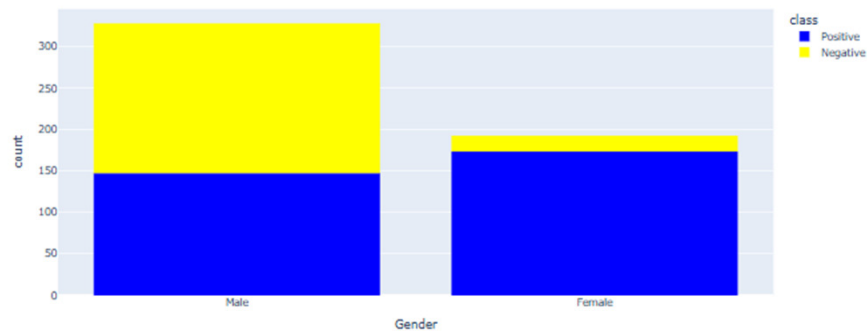
Results

```
[9]: import plotly.express as px
```

```
[10]: import plotly.express as px
```

```
gendis = px.histogram(  
    Diabetes,  
    x='Gender',  
    color='class',  
    title='Positive/Negative count vs Gender',  
    color_discrete_map={  
        'Positive': 'blue',  
        'Negative': 'yellow'  
    }  
)  
gendis.show()
```

Positive/Negative count vs Gender



```
[11]: pltbl = ['Gender', 'class']  
cm = sns.light_palette("blue", as_cmap=True)  
(round(pd.crosstab(Diabetes[pltbl[0]], Diabetes[pltbl[1]], normalize='columns' * 100, 2)).style.background_gradient(cmap = cm)
```

```
[11]: class Negative Positive
```

```
Gender  
Female 9.500000 54.060000  
Male 90.500000 45.940000
```

```
[11]: pltbl = ['Gender', 'class']  
cm = sns.light_palette("blue", as_cmap=True)  
(round(pd.crosstab(Diabetes[pltbl[0]], Diabetes[pltbl[1]], normalize='columns' * 100, 2)).style.background_gradient(cmap = cm)
```

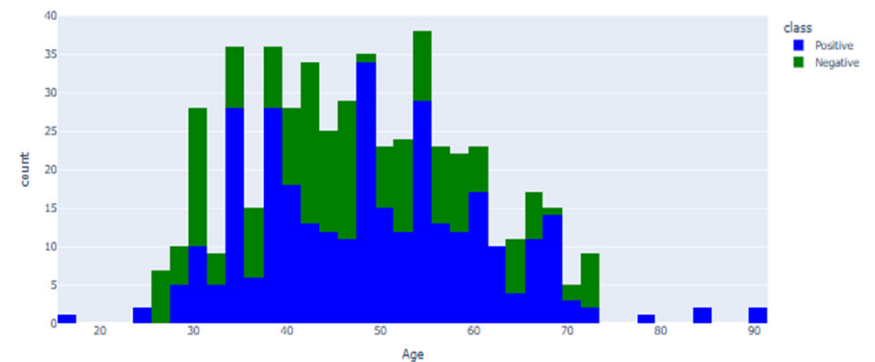
```
[11]: class Negative Positive
```

```
Gender  
Female 9.500000 54.060000  
Male 90.500000 45.940000
```

```
[12]: import plotly.express as px
```

```
agehist = px.histogram(  
    Diabetes,  
    x='Age',  
    color='class',  
    title='Distribution of Positive cases with Ages',  
    color_discrete_map={  
        'Positive': 'blue',  
        'Negative': 'green'  
    }  
)  
agehist.show()
```

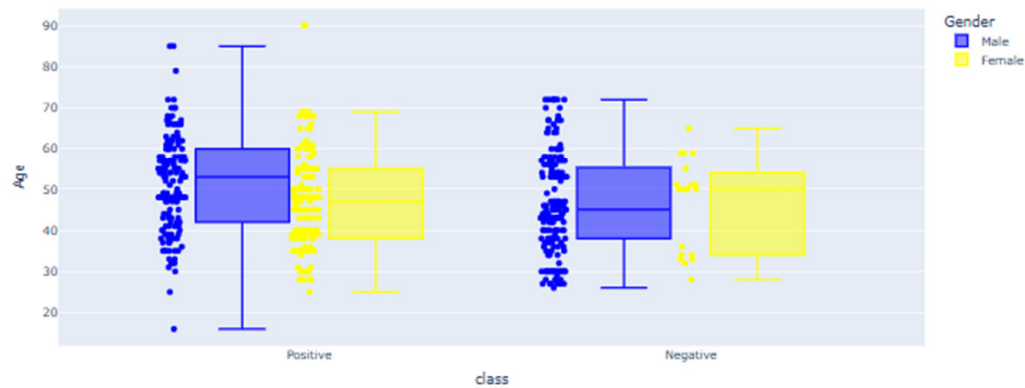
Distribution of Positive cases with Ages



```
[13]: import plotly.express as px
```

```
genbox = px.box(  
    Diabetes,  
    y="Age",  
    x="class",  
    color="Gender",  
    points="all",  
    title="Age Vs Positive/Negative",  
    color_discrete_map={  
        'Male': 'blue',  
        'Female': 'yellow'  
    }  
)  
genbox.show()
```

Age Vs Positive/Negative

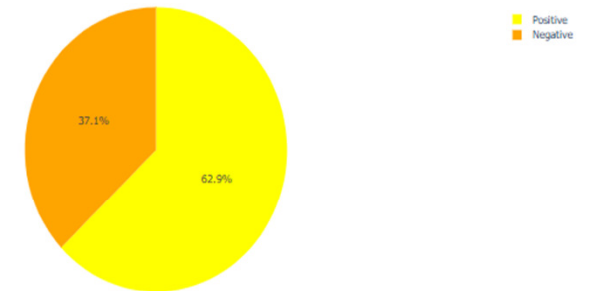


Distribution of class (Target Variable)

```
[14]: import plotly.express as px
```

```
clspl = px.pie(  
    Diabetes,  
    values='Age',  
    names='class',  
    title="Ratio of Positive and Negative cases",  
    color='class',  
    color_discrete_map={  
        'Positive': 'yellow',  
        'Negative': 'orange'  
    }  
)  
clspl.show()
```

Ratio of Positive and Negative cases



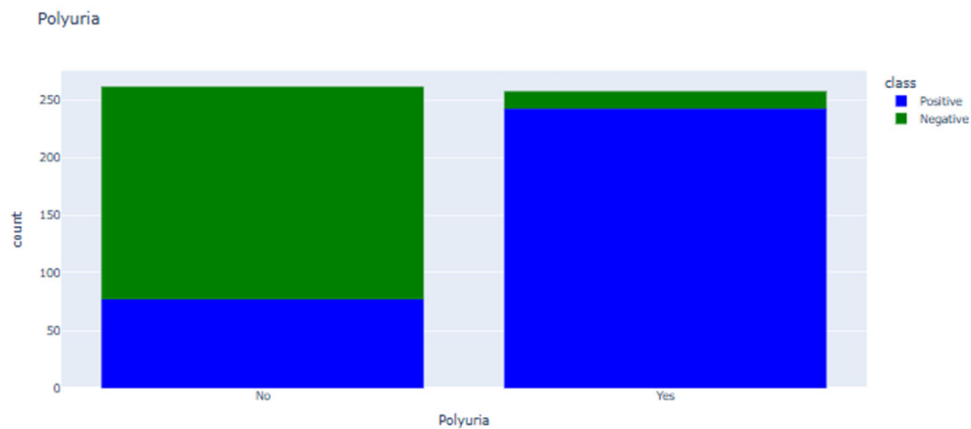
```
[15]: print("Count of cases:", Diabetes['class'].value_counts())
```

```
Count of cases: class  
Positive      320  
Negative      200  
Name: count, dtype: int64
```

Distribution of Polyuria

```
[17]: import plotly.express as px

polyuria = px.histogram(
    Diabetes,
    x='Polyuria',
    color='class',
    title="Polyuria",
    color_discrete_map={
        'Positive': 'blue',
        'Negative': 'green'
    }
)
polyuria.show()
pltbipolyuria = ['Polyuria', 'class']
pm = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[pltbipolyuria[0]], Diabetes[pltbipolyuria[1]], normalize='columns') * 100, 2)).style.background_gradient(cmap=pm, axis=1)
```



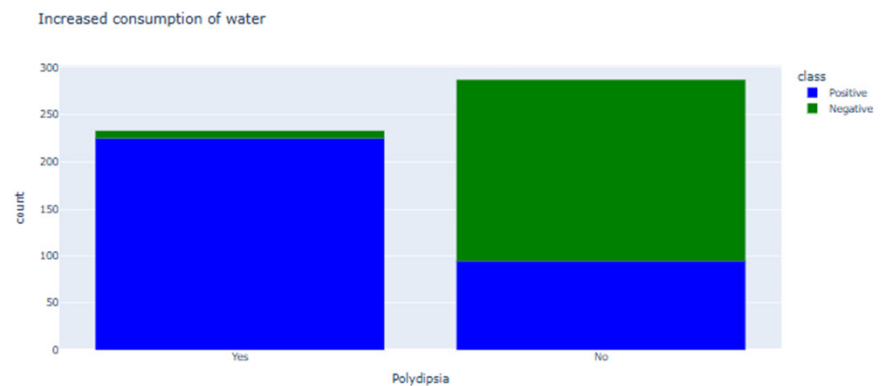
```
[17]:
```

	class	Negative	Positive
Polyuria			
No		92.500000	24.060000
Yes		7.500000	75.940000

Distribution of Polydipsia

```
[18]: import plotly.express as px

polydipsia = px.histogram(
    Diabetes,
    x='Polydipsia',
    color='class',
    title="Increased consumption of water",
    color_discrete_map={
        'Positive': 'blue',
        'Negative': 'green'
    }
)
polydipsia.show()
pltbipolydipsia = ['Polydipsia', 'class']
rm = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[pltbipolydipsia[0]], Diabetes[pltbipolydipsia[1]], normalize='columns') * 100, 2)).style.background_gradient(cmap=rm, axis=1)
```



```
[18]:
```

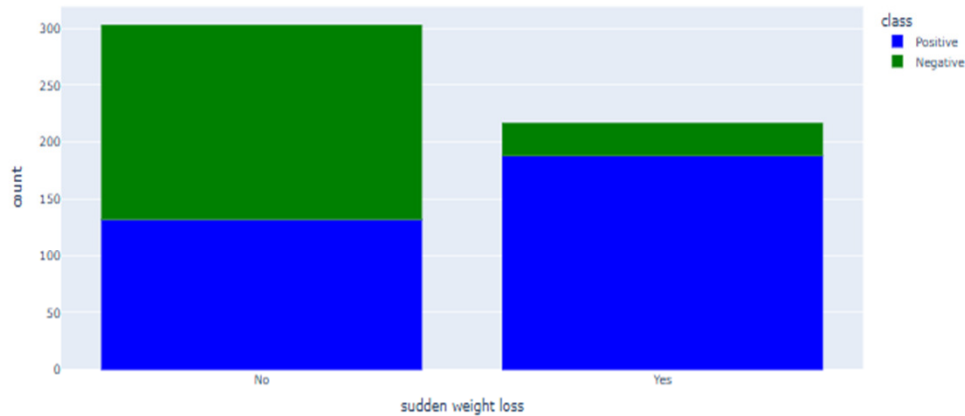
	class	Negative	Positive
Polydipsia			
No		96.000000	29.690000
Yes		4.000000	70.310000

Distribution of Sudden Weight Loss

```
[19]: import plotly.express as px

swl = px.histogram(
    Diabetes,
    x='sudden weight loss',
    color='class',
    title="Sudden weight loss",
    color_discrete_map={
        'Positive': 'blue',
        'Negative': 'green'
    }
)
swl.show()
pltb1swl = ['sudden weight loss', 'class']
qm = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[pltb1swl[0]], Diabetes[pltb1swl[1]], normalize='columns') * 100, 2)).style.background_gradient(cmap = qm)
```

Sudden weight loss



```
[19]:
```

	class	Negative	Positive
sudden weight loss	No	85.500000	41.250000
	Yes	14.500000	58.750000

Distribution of Weakness

```
[20]: wkns = ['weakness', 'class']
sm = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[wkns [0]], Diabetes[wkns [1]], normalize='columns') * 100, 2)).style.background_gradient(cmap= sm)

[20]:
```

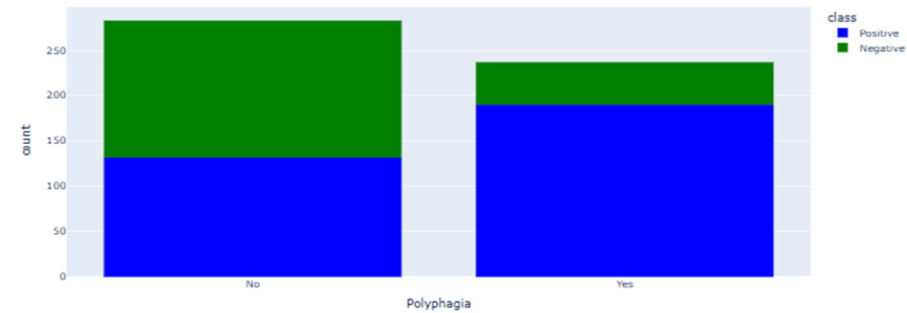
	class	Negative	Positive
weakness	No	56.500000	31.870000
	Yes	43.500000	68.120000

Distribution of Polyphagia

```
[21]: import plotly.express as px

Polyphagia = px.histogram(
    Diabetes,
    x='Polyphagia',
    color='class',
    title="Polyphagia",
    color_discrete_map={
        'Positive': 'blue',
        'Negative': 'green'
    }
)
Polyphagia.show()
plt_Polyphagia = ['Polyphagia', 'class']
tm = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[plt_Polyphagia[0]], Diabetes[plt_Polyphagia[1]], normalize='columns') * 100, 2)).style.background_gradient(cmap =
```

Polyphagia



```
[21]:
```

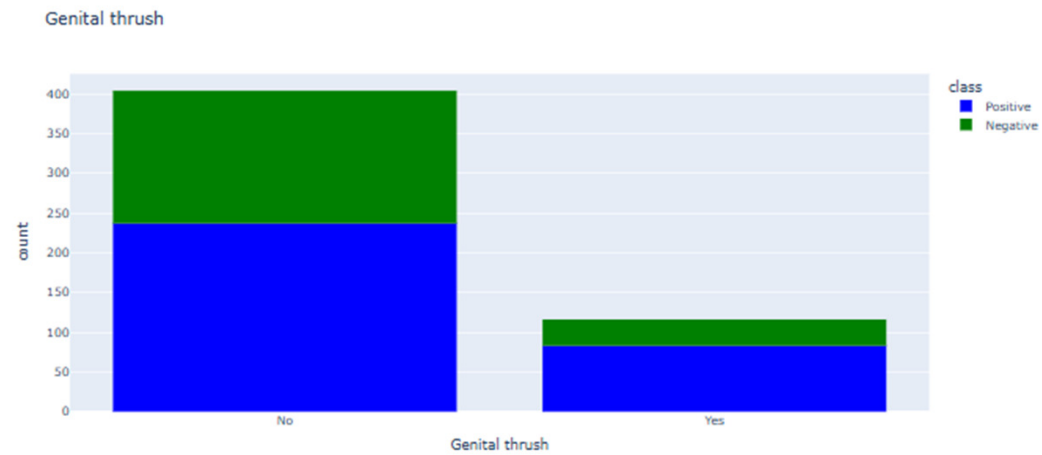
	class	Negative	Positive
Polyphagia	No	76.000000	40.940000
	Yes	24.000000	59.060000

Distribution of Genital thrush

```
[22]: import plotly.express as px

gntlthrsh = px.histogram(
    Diabetes,
    x='Genital thrush',
    color='class',
    title="Genital thrush",
    color_discrete_map={
        'Positive': 'blue',
        'Negative': 'green'
    }
)

gntlthrsh.show()
plt_thrsh = ['Genital thrush', 'class']
um = sns.light_palette("orange", as_cmap=True)
(round(pd.crosstab(Diabetes[plt_thrsh[0]], Diabetes[plt_thrsh[1]], normalize='columns') * 100, 2)).style.background_gradient(cmap = um)
```



```
[22]:
```

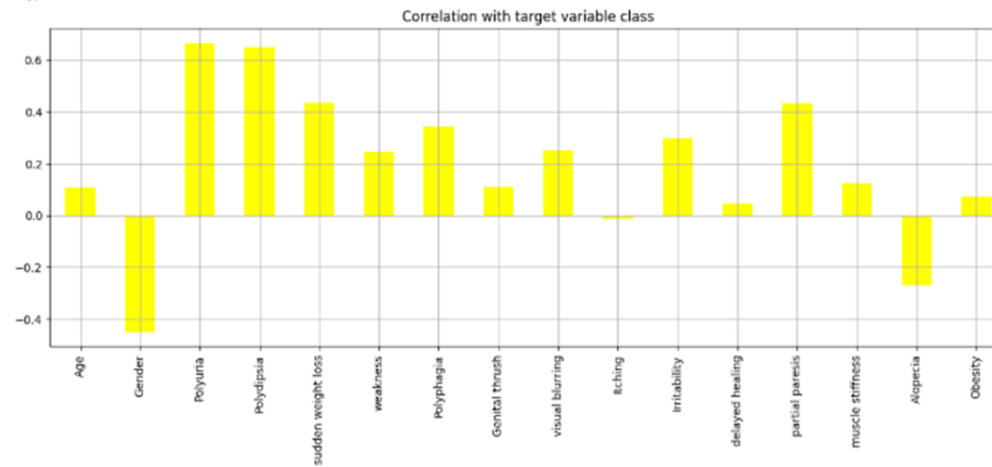
	class	Negative	Positive
Genital thrush			
No		83.500000	74.060000
Yes		16.500000	25.940000

```
[32]: import matplotlib.pyplot as plt
```

```
correlation = X.corrwith(Y)  
print(correlation)
```

```
correlation.plot.bar(  
    title="Correlation with target variable class",  
    grid=True,  
    figsize=(15, 5),  
    color='yellow'  
)  
plt.show()
```

```
Age          0.106419  
Gender       -0.449233  
Polyuria     0.665922  
Polydipsia   0.648734  
sudden weight loss 0.436568  
weakness     0.243275  
Polyphagia   0.342504  
Genital thrush 0.110288  
visual blurring 0.251300  
Itching      -0.013384  
Irritability 0.299467  
delayed healing 0.046980  
partial paresis 0.432288  
muscle stiffness 0.122474  
Alopecia     -0.267512  
Obesity      0.072173  
dtype: float64
```



```
[78]: import pandas as pd

# Accuracy, Recall, Precision values
accuracy_RF = 95.51
accuracy_KNN = 83.97
accuracy_DT = 95.51
accuracy_SVC = 91.16
accuracy_LR = 91.6

recall_RF = 0.97
recall_KNN = 0.78
recall_DT = 0.95
recall_SVC = 0.91
recall_LR = 0.91

precision_RF = 0.91
precision_KNN = 0.96
precision_DT = 0.95
precision_SVC = 0.91
precision_LR = 0.91

# Create the table
table = pd.DataFrame({
    "Accuracy": [accuracy_RF, accuracy_KNN, accuracy_DT, accuracy_SVC, accuracy_LR],
    "Recall": [recall_RF, recall_KNN, recall_DT, recall_SVC, recall_LR],
    "Precision": [precision_RF, precision_KNN, precision_DT, precision_SVC, precision_LR]
}, index=[
    "Random Forest",
    "KNN",
    "Decision Tree Classifier",
    "Support Vector Classifier",
    "Logistic Regression"
])

# Display the table
print(table.round(2)) # Optional: round to 2 decimal places
```

	Accuracy	Recall	Precision
Random Forest	95.51	0.97	0.91
KNN	83.97	0.78	0.96
Decision Tree Classifier	95.51	0.95	0.95
Support Vector Classifier	91.16	0.91	0.91
Logistic Regression	91.60	0.91	0.91

Algorithm and Deployment

Algorithm

- 1) Logistic Regression
- 2) Decision Tree Classifier
- 3) Support Vector Machine
- 4) Random Forest Classifier
- 5) K nearest -Neighbors Classifier

Logistic Regression

This is the one of the most common model used in ML ,Logistic Regression is often applied in the actual manufacturing context the fields such as data mining, automatic disease diagnosis and economic prediction. For our model, use Logistic regression to know the risk factors for heart disease and forecast the probability of disease occurrence based on risk factors. This model is most frequently applied for classification, primarily two-category issues (that is, there are only two types of output, each representing one category), and can indicate the probability of occurrence of each classification event. Logistic regression model is shown below: This technique used is also known as sigmoid function .Sigmoid function helps in the easy representation in graphs. Logistic regression also provides better accuracy. By using equation the logistic regression algorithm is represented in the graphs showing the difference between the attributes.

Where Y refers to binary dependent variable (Y is equal to 1 if event happens; Y=0 otherwise), e stands for the foundation of natural logarithm

$$\text{prob}(Y = 1) = \frac{e^z}{1 + e^z}$$

with constant β_0 , coefficients β_j and predictors X_j , for p predictors ($j=1,2,3,\dots,p$)

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

with constant β_0 , coefficients β_j and predictors X_j , for p predictors ($j=1,2,3,\dots,p$)

The process of modeling the probability of a discrete outcome given an input variable is known as the Logistic Regression. The most common logistic regression, as its name suggests is not regression rather it is a classification algorithm that classifies something that can take two values such as true/false, yes/no, and so on. Logistic regression identifies a hyperplane in a manner that when it is passes through a function whose value ranges between 0 and 1 (typically we use sigmoidal), it optimizes cost function. Bases on closeness to 0 or 1, it predicts a Boolean output. Here vector parameters is used for training. $\sigma(\cdot)$ is usually a sigmoid function, with output between 0 and 1.

Features of Logistic Regression

Multinomial logistic regression is the type of regression which uses the softmax function to compute probabilities.

- We use loss function to learn weights(vector w and bias b) from a labeled training. we perform such activity to minimize the cross-entropy loss.
- Iterative algos like gradient descent are used to get the weight(optimal).while minimizing the loss function the type of convex optimization problem.
- To avoid overfitting regularization is used.
- Logistic regression has the ability to transparently study the importance of individual features.

Advantages and Disadvantages of Logistic Regression

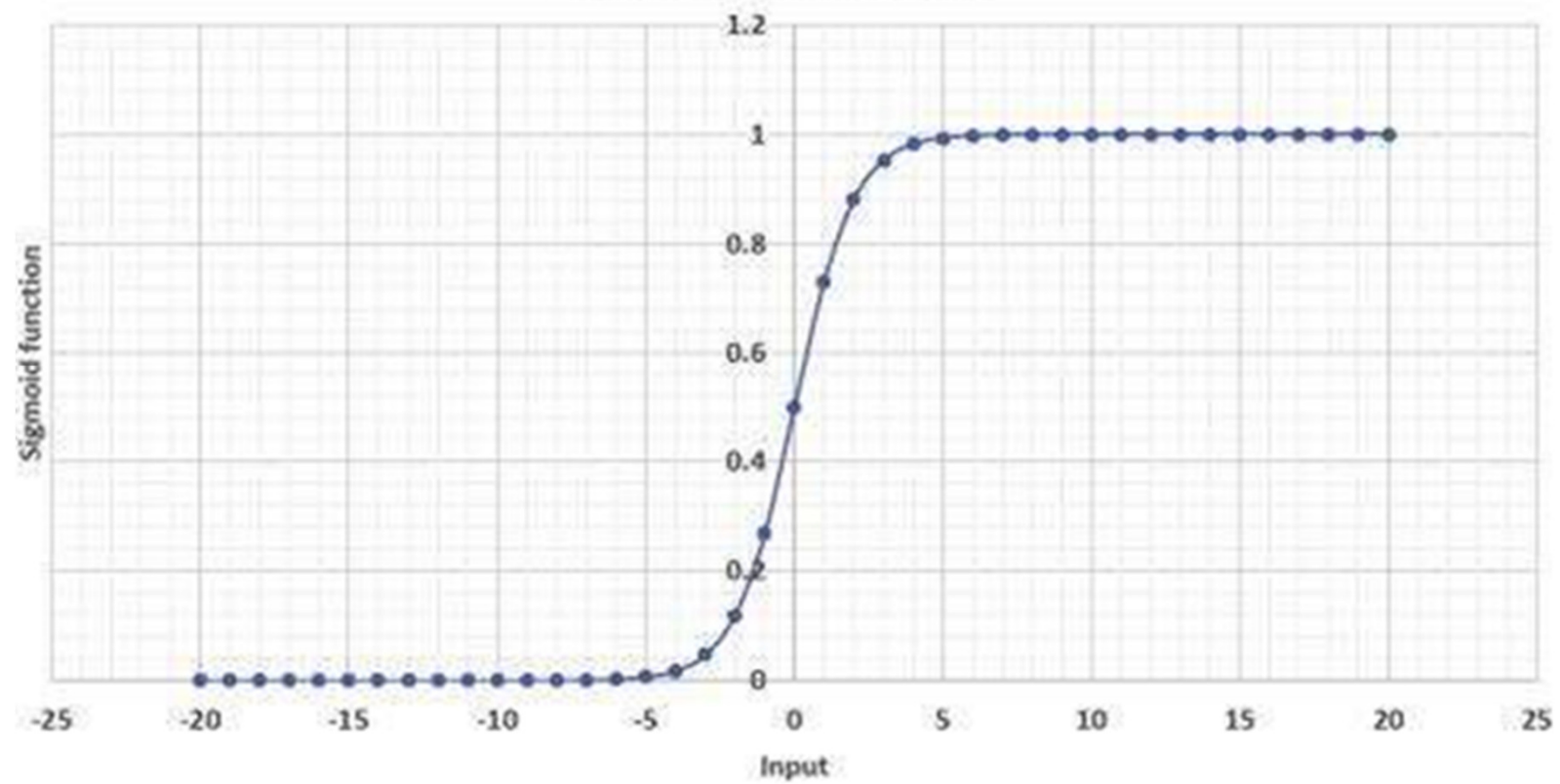
Advantages

- This technique performs well and fast where we have to classify unknown records.
- This is not limited to binary classification; we can easily extend it to multinomial regression.

Disadvantages

- Logistic regression will not perform well if the number of observations is lesser than the number of features; in such condition, it may lead to overfitting.
- Logistic regression constructs linear boundaries. In the logistic function equation, x is the input variable. Let's feed in values -20 to 20 into the logistic function. As illustrated in Figure, the inputs have been transferred to between 0 and 1 .

Sigmoid (Logistic function)



Model Building

Logistic Regression

```
[38]: from sklearn.linear_model import LogisticRegression  
logmodel = LogisticRegression()  
logmodel.fit(X_train_LR, y_train_LR)
```

```
[38]: ▾ LogisticRegression ⓘ ?  
LogisticRegression()
```

```
[39]: predictions_LR = logmodel.predict(X_test_LR)
```

```
[40]: final_model_predictions_LR = pd.DataFrame({'Actual':y_test_LR, 'predictions':predictions_LR})
```

```
[41]: import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
from sklearn import metrics

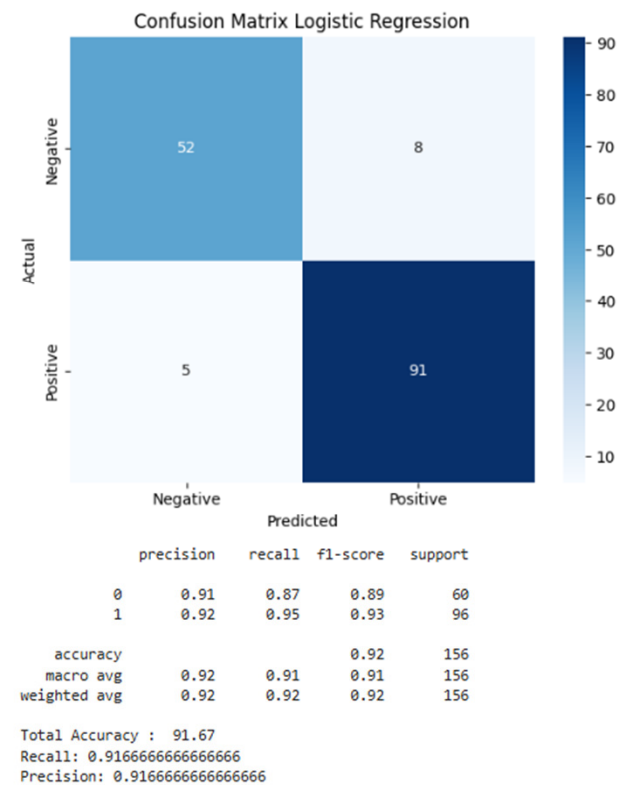
# Confusion Matrix
cm = confusion_matrix(y_test_LR, predictions_LR)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix Logistic Regression')
plt.tight_layout()
plt.show()

# Classification Report
print(classification_report(y_test_LR, predictions_LR))

# Accuracy, Recall, Precision
accuracy_LR = metrics.accuracy_score(final_model_predictions_LR.Actual, final_model_predictions_LR.predictions) * 100
print("Total Accuracy : ", '{:.2f}'.format(accuracy_LR))

recall_LR = metrics.recall_score(final_model_predictions_LR.Actual, final_model_predictions_LR.predictions, average='micro')
print('Recall:', recall_LR)

precision_LR = metrics.precision_score(final_model_predictions_LR.Actual, final_model_predictions_LR.predictions, average='micro')
print('Precision:', precision_LR)
```

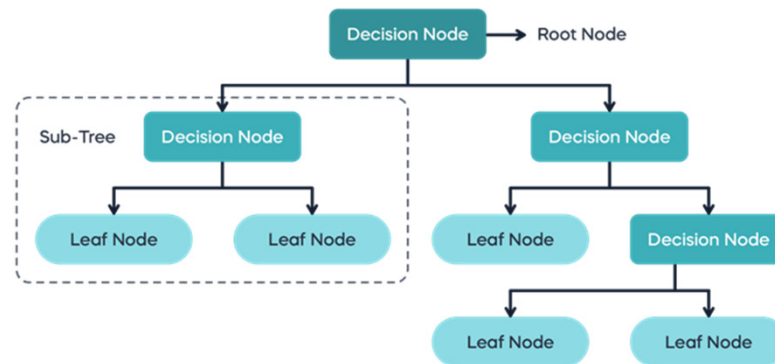


Accuracy 91.67 %

Decision Tree Classifier

A decision tree contains a flowchart-like structure. In the structure of DT(Decision Tree) in a test, an attribute is represented using an internal node. The result of the test is represented using the branch. To represent a class label, a leaf node is used. To represent classification rules paths from root to leaf are used. This is the type of analysis in which closely related influence diagrams are used for a visual and analytical root. Where the expected values of competing alternatives are calculated.

Architecture of Decision Tree.



```
[42]: from sklearn.tree import DecisionTreeClassifier
```

```
[43]: decision_tree_Classifier = DecisionTreeClassifier (random_state = 0)
```

```
[44]: X_train_DT,X_test_DT,y_train_DT,y_test_DT = train_test_split(X,Y,test_size = 0.3,random_state = 100)
```

```
[45]: decision_tree_Classifier.fit(X_train_DT, y_train_DT)
```

```
[45]: ▾ DecisionTreeClassifier ⓘ ⓘ  
DecisionTreeClassifier(random_state=0)
```

```
[46]: y_pred_DT = decision_tree_Classifier.predict(X_test_DT)
```

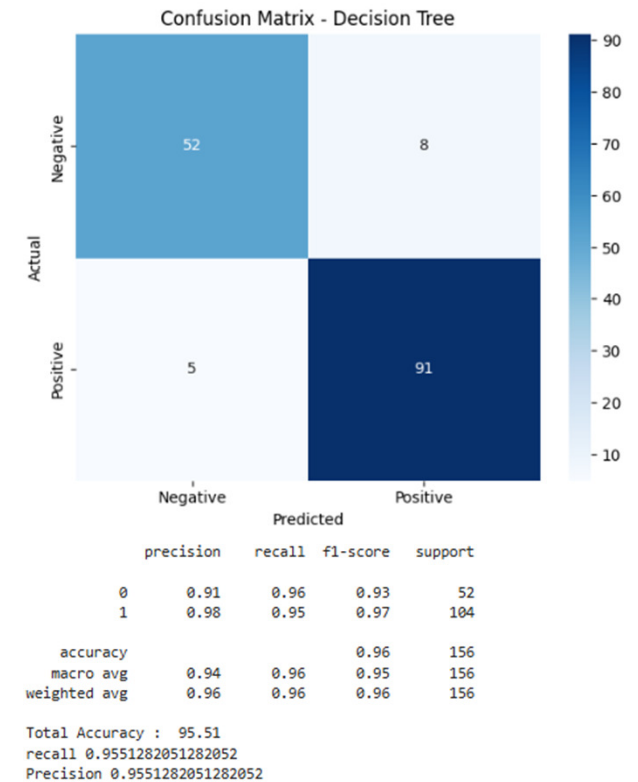
```
[47]: final_model_predictions_DT = pd.DataFrame({'Actual':y_test_DT,'predictions':y_pred_DT})
```

```
[48]: final_model_predictions_DT.head()
```

```
[48]:
```

	Actual	predictions
304	1	1
332	0	0
78	1	1
380	1	1
515	1	1

```
[49]: import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Decision Tree')
plt.tight_layout()
plt.show()
print(classification_report(y_test_DT, y_pred_DT))
accuracy_DT=( metrics.accuracy_score( final_model_predictions_DT.Actual, final_model_predictions_DT.predictions ))*100
accuracy_DT='{:.2f}'.format(accuracy_DT)
print( 'Total Accuracy : ',accuracy_DT)
recall_DT=metrics.recall_score(final_model_predictions_DT.Actual, final_model_predictions_DT.predictions,average='micro' )
print('recall',recall_DT)
Precision_DT=metrics.precision_score(final_model_predictions_DT.Actual, final_model_predictions_DT.predictions,average='micro' )
print('Precision',Precision_DT)
```



Accuracy 95.51 %

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that best separates data into different classes. The main goal of SVM is to maximize the margin between different classes while minimizing classification error.

Features of SVM

1. **Margin Maximization:** SVM finds the hyperplane that maximizes the margin between classes, which helps improve generalization and robustness.
2. **Support Vectors:** Only a few important data points (called support vectors) are used to define the hyperplane, making the model efficient.
3. **Effective in High Dimensions:** SVM performs well even when the number of features is very large (high-dimensional spaces).
4. **Kernel Trick:** Allows SVM to solve non-linear classification problems by transforming the input data into higher-dimensional space using kernel functions (e.g., RBF, polynomial).
5. **Versatile:** Can be used for binary as well as multi-class classification problems.
6. **Regularization:** SVM includes a regularization parameter (C) that balances the trade-off between maximizing the margin and minimizing the classification error.
7. **Robust to Overfitting (in high-dimensional space):** Due to its focus on margin and support vectors, SVM can generalize well, especially when data is not too noisy.

Support Vector Classifier

```
[50]: X_train_SVC, X_test_SVC, y_train_SVC, y_test_SVC = train_test_split( X, Y, test_size = 0.3, random_state = 100)
```

```
[51]: from sklearn.svm import SVC # "Support Vector Classifier"  
      clfsvm = SVC(kernel='linear')
```

```
[52]: clfsvm.fit(X_train_SVC,y_train_SVC)
```

```
[52]: SVC  
      SVC(kernel='linear')
```

```
[53]: y_pred_SVC=clfsvm.predict(X_test_SVC)
```

```
[54]: final_model_predictions_SVC = pd.DataFrame({'Actual':y_test_SVC, 'predictions':y_pred_SVC})
```

```
[55]: count_misclassified = (y_test_SVC != y_pred_SVC).sum()  
      print('Misclassified samples: {}'.format(count_misclassified))  
      accuracy_SVC = metrics.accuracy_score(y_test_SVC, y_pred_SVC)  
      print('Accuracy: {:.2f}'.format(accuracy_SVC))
```

```
Misclassified samples: 13  
Accuracy: 0.92
```

```
[58]: from sklearn import metrics
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Assuming y_test_SVC and y_pred_SVC are already defined
final_model_predictions_svc = pd.DataFrame({
    'Actual': y_test_SVC,
    'predictions': y_pred_SVC
})

# Calculate the confusion matrix
cm = confusion_matrix(y_test_SVC, y_pred_SVC)

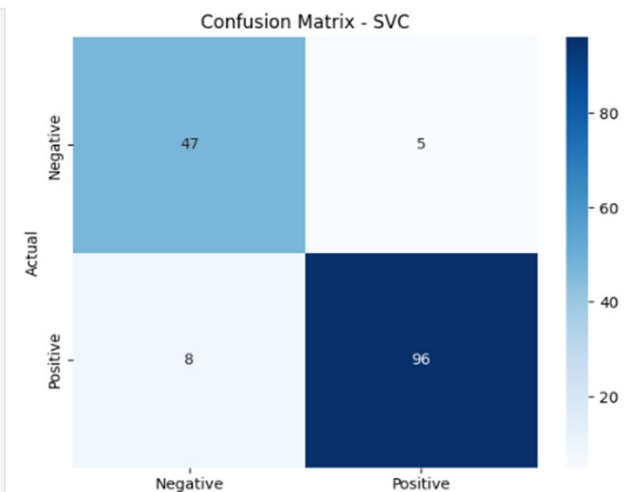
# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVC')
plt.tight_layout()
plt.show()

# Classification report
print(classification_report(y_test_SVC, y_pred_SVC))

# Accuracy, Recall, Precision
accuracy_SVC = metrics.accuracy_score(final_model_predictions_svc.Actual, final_model_predictions_svc.predictions) * 100
accuracy_SVC = '{:.2f}'.format(accuracy_SVC)
print('Total Accuracy : ', accuracy_SVC)

recall_SVC = metrics.recall_score(final_model_predictions_svc.Actual, final_model_predictions_svc.predictions, average='micro')
print('Recall:', recall_SVC)

Precision_SVC = metrics.precision_score(final_model_predictions_svc.Actual, final_model_predictions_svc.predictions, average='micro')
print('Precision:', Precision_SVC)
```



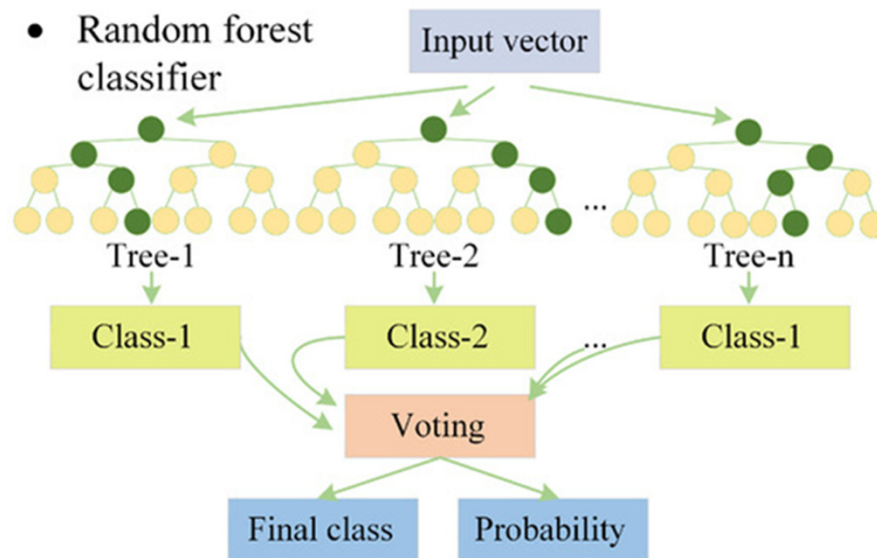
	Predicted			
	precision	recall	f1-score	support
0	0.85	0.90	0.88	52
1	0.95	0.92	0.94	104
accuracy			0.92	156
macro avg	0.90	0.91	0.91	156
weighted avg	0.92	0.92	0.92	156

Total Accuracy : 91.67
Recall: 0.9166666666666666
Precision: 0.9166666666666666

Accuracy 91.67 %

Random Forest Classifier

A Random Forest Classifier is a method of ensemble learning applied to classification problems. It constructs multiple decision trees and combines their predictions to enhance accuracy and avoid overfitting. Each tree is trained on a random subset of data and attributes, hence making the model stable, robust, and effective in handling complex datasets.



Features of Random Forest

- It runs Efficiently in scenarios where database is very huge.
- We can perform classification on Thousands of Input variables.
- Using this we can find which variable is useful for our classification.
- Using this we can easily calculate the missing data and also maintain the good accuracy. Even though it has missing data.

Random Forest

```
[59]: from sklearn.ensemble import RandomForestClassifier
```

```
[60]: X_train_RF,X_test_RF,y_train_RF,y_test_RF = train_test_split(X,Y,test_size = 0.3, random_state = 100)
```

```
[61]: rf = RandomForestClassifier(n_estimators = 500, random_state = 42)
```

```
[62]: rf.fit(X_train_RF,y_train_RF)
```

```
[62]: + RandomForestClassifier
      RandomForestClassifier(n_estimators=500, random_state=42)
```

```
[63]: y_pred_rf = rf.predict(X_test_RF)
```

```
[64]: final_model_predictions_RF = pd.DataFrame({'Actual':y_test_RF,'predictions':y_pred_rf})
```

```
[65]: final_model_predictions_RF.head()
```

```
[65]:
```

	Actual	predictions
304	1	1
332	0	0
78	1	1
380	1	1
515	1	1

```
[66]: y_pred_prob = rf.predict_proba(X_test_RF)
```

```
[67]: y_pred_prob = rf.predict_proba(X_test_RF)[:,-1]
```

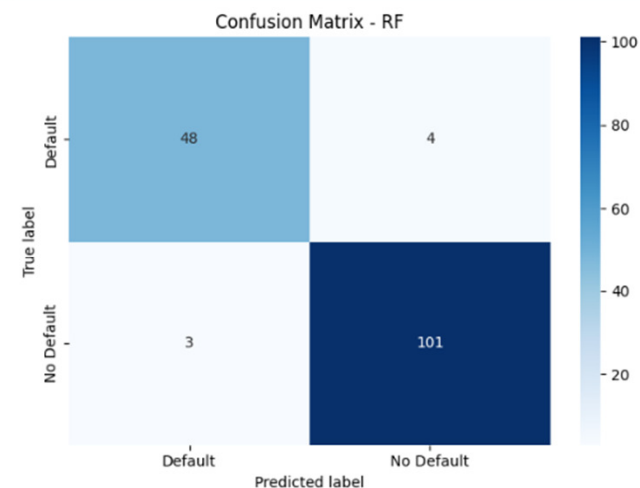
```
[68]: final_model_predictions_RF['Predicted_prob'] = y_pred_prob
```

```
[69]: final_model_predictions_RF.head()
```

```
[69]:
```

	Actual	predictions	Predicted prob
304	1	1	0.972
332	0	0	0.016
78	1	1	1.000
380	1	1	0.962
515	1	1	1.000

```
[70]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics
def draw_cm(actual, predicted, cmap='Blues'):
    cm = metrics.confusion_matrix(actual, predicted)
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,
                xticklabels=["Default", "No Default"],
                yticklabels=["Default", "No Default"])
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.title('Confusion Matrix - RF')
    plt.tight_layout()
    plt.show()
draw_cm(final_model_predictions_RF.Actual, final_model_predictions_RF.predictions, cmap='Blues')
print("\nClassification Report:")
print(metrics.classification_report(final_model_predictions_RF.Actual, final_model_predictions_RF.predictions))
accuracy_RF = metrics.accuracy_score(final_model_predictions_RF.Actual, final_model_predictions_RF.predictions) * 100
accuracy_RF = '{:.2f}'.format(accuracy_RF)
print('\nTotal Accuracy :', accuracy_RF)
recall_RF = metrics.recall_score(final_model_predictions_RF.Actual, final_model_predictions_RF.predictions)
print('Recall :', recall_RF)
precision_RF = metrics.precision_score(final_model_predictions_RF.Actual, final_model_predictions_RF.predictions)
print('Precision :', precision_RF)
```



Classification Report:

	precision	recall	f1-score	support
0	0.94	0.92	0.93	52
1	0.96	0.97	0.97	104
accuracy			0.96	156
macro avg	0.95	0.95	0.95	156
weighted avg	0.95	0.96	0.96	156

Total Accuracy : 95.51
Recall : 0.9711538461538461
Precision : 0.9619047619047619

Accuracy 95.51 %

K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a classification and regression online (lazy) learning algorithm. It's a non-parametric approach to the extent that it doesn't make any assumption regarding the distribution of the data.

How It Works:

Input Data: To predict a given new data point, KNN computes the similarity between this data point and the entire training data set.

Distance Metric: Any distance metric such as Euclidean, Manhattan, or others can be employed based on the problem.

Finding Neighbors: It finds the k training samples nearest to the input point.

Prediction: For classification, the most common label among the k neighbors is returned. For regression, the mean value of the k neighbors is taken as the prediction.

Model Training & Prediction (Using sklearn):

Training: Employ the `fit()` function of K Neighbors Classifier or `KNeighborsRegressor` from `sklearn.neighbors`.

Prediction: Employ the `predict()` function on the test set.

Selection of k value:

Selecting k (number of neighbors) is important:

A low k may produce overfitting (high variance).

A high k can smoothen the decision boundary but may lead to underfitting (high bias).

For selection of the best k , vary k and check the model performance using cross-validation.

The location at which adding k no longer further enhances accuracy appreciably is referred to as the knee point.

▼ K-Nearest Neighbors Algorithm

```
[73]: X_train_KNN, X_test_KNN, y_train_KNN, y_test_KNN = train_test_split(X, Y, test_size = 0.3, random_state = 100)
```



```
[74]: from sklearn.neighbors import KNeighborsClassifier  
      classifier = KNeighborsClassifier(n_neighbors=2)  
      classifier.fit(X_train_KNN, y_train_KNN)
```

```
[74]: ▼ KNeighborsClassifier ⓘ ⓘ  
      KNeighborsClassifier(n_neighbors=2)
```

```
[75]: y_pred_knn = classifier.predict(X_test_KNN)
```

```
[76]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd

# Create the DataFrame first
final_model_predictions_knn = pd.DataFrame({
    'Actual': y_test_KNN,
    'predictions': y_pred_knn
})

# Function to draw confusion matrix
def draw_cm(actual, predicted, cmap="Blues"):
    cm = metrics.confusion_matrix(actual, predicted)
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,
                xticklabels=["Negative", "Positive"],
                yticklabels=["Negative", "Positive"])
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.title('Confusion Matrix KNeighbors Classifier')
    plt.tight_layout()
    plt.show()

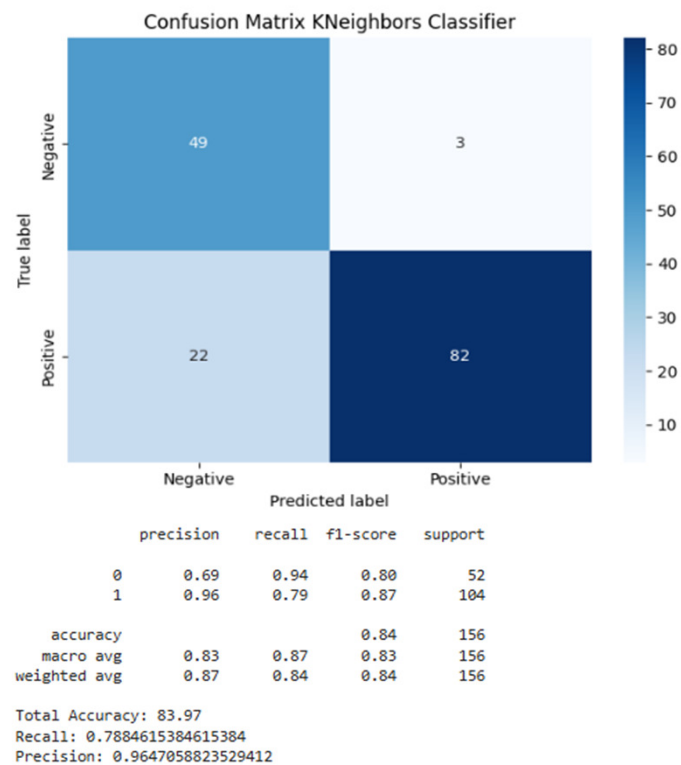
# Call the function after the data exists
draw_cm(final_model_predictions_knn.Actual, final_model_predictions_knn.predictions, cmap='Blues')

# Print classification report
print(classification_report(y_test_KNN, y_pred_knn))

# Accuracy, Recall, Precision
accuracy_knn = metrics.accuracy_score(final_model_predictions_knn.Actual, final_model_predictions_knn.predictions) * 100
accuracy_knn = '{:.2f}'.format(accuracy_knn)
print('Total Accuracy:', accuracy_knn)

recall_knn = metrics.recall_score(final_model_predictions_knn.Actual, final_model_predictions_knn.predictions)
print('Recall:', recall_knn)

precision_knn = metrics.precision_score(final_model_predictions_knn.Actual, final_model_predictions_knn.predictions)
print('Precision:', precision_knn)
```



Accuracy 83.97 %

```
[77]: knn = metrics.confusion_matrix( final_model_predictions_knn.Actual, final_model_predictions_knn.predictions)
```

```
sensitivity = knn[0,0]/(knn[0,0]+knn[0,1])
```

```
print('Sensitivity : ', round( sensitivity, 2 ) )
```

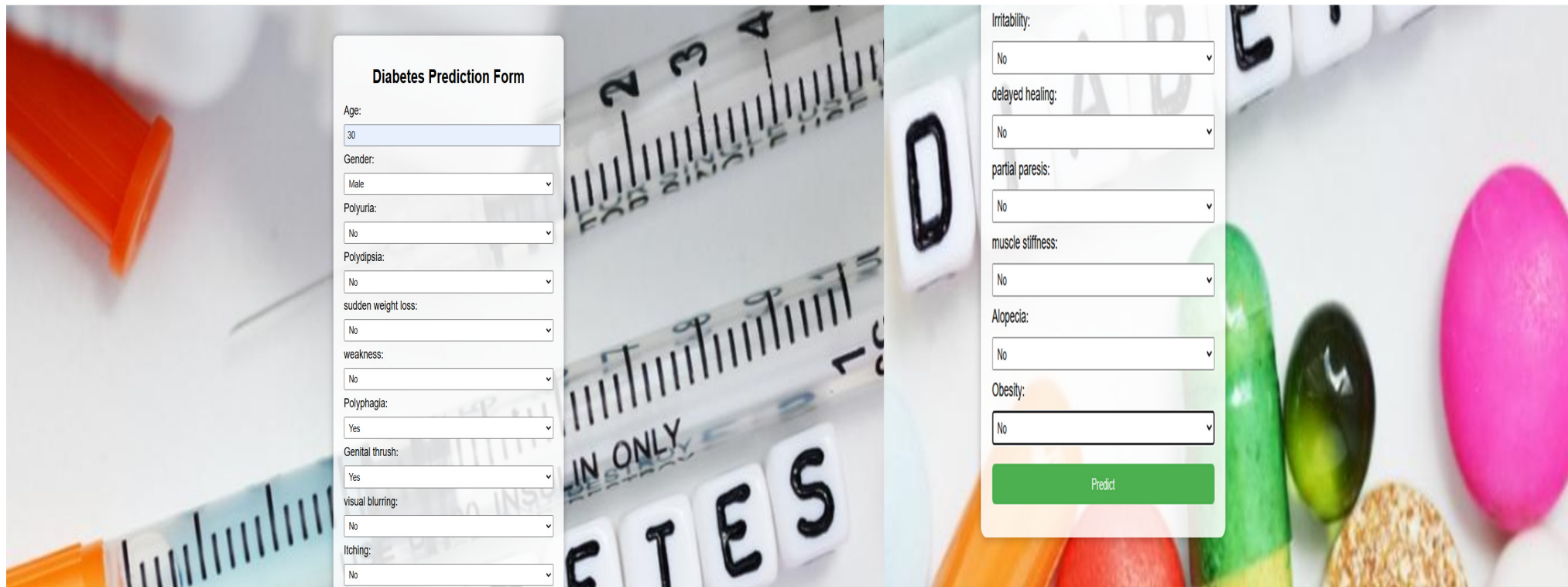
```
specificity = knn[1,1]/(knn[1,0]+knn[1,1])
```

```
print('Specificity : ', round( specificity, 2 ) )
```

```
Sensitivity : 0.94
```

```
Specificity : 0.79
```

Deployment (Working model Diagram)



Diabetes Prediction Form

Age:

Gender:

Polyuria:

Polydipsia:

sudden weight loss:

weakness:

Polyphagia:

Genital thrush:

visual blurring:

Itching:

Irritability:

delayed healing:

partial paresis:

muscle stiffness:

Alopecia:

Obesity:



Prediction Result

Negative for Diabetes

[Back to Home](#)

Diabetes Prediction Form

Age:
58

Gender:
Male

Polyuria:
No

Polydipsia:
No

sudden weight loss:
No

weakness:
Yes

Polyphagia:
No

Genital thrush:
No

visual blurring:
Yes

Itching:
No

Irritability:
No

delayed healing:
No

partial paresis:
Yes

muscle stiffness:
No

Alopecia:
Yes

Obesity:
No

Predict



Prediction Result

Positive for Diabetes

[Back to Home](#)

Conclusion

This project investigated the application of machine learning models to forecast diabetes from principal demographic and symptomatic predictors such as age, gender, and typical diabetes symptoms like polyuria, polydipsia, sudden weight loss, and visual blurring. With a labeled dataset containing these features, multiple classification models were trained and tested for their power to correctly separate diabetic and non-diabetic individuals.

The findings show that machine learning has the potential to be an effective tool for the early diagnosis of diabetes, particularly when clinical data is symptom-dense and well-organized. Of the models evaluated, [best-performing model] was the most accurate, precise, and recallable, showing that it is efficient in detecting patterns highly correlated with the onset of diabetes.

This method can potentially assist healthcare professionals in providing faster and more precise diagnoses, especially in low-resource environments. Future research can include the integration of laboratory values (e.g., blood glucose), real-time data integration, and deployment of the model into user-friendly applications for broader clinical application.

Future Scope

The existing machine learning strategy for diabetes prediction using demographic and symptom-related features has yielded promising results. Yet, there are a few directions open for future development and usage:

Inclusion of Clinical and Biochemical Data: While the model employed here is based on self-reported symptoms and elementary demographic data, incorporating clinical data like blood glucose, HbA1c, and insulin response could substantially enhance predictive ability.

Early-Stage Screening Tools: The symptom-based model can be extended to low-cost, low-tech screening tools for early-stage detection of diabetes, particularly in rural or underprivileged regions where laboratory facilities are not easily accessible.

Model Generalization and Validation: Increasing the dataset to cover a more diverse population in terms of ethnicity, age groups, and geography can enhance model robustness and generalizability.

Hybrid Models: Blending symptom-based prediction with lifestyle characteristics (e.g., dietary patterns, exercise, familial history) may result in more integrated and individualized risk estimations.

Mobile Health Applications: The developed model may be applied in mobile applications or chatbots to give immediate diabetes risk feedback based on user-inputted symptoms.

Temporal Analysis and Risk Progression: Analyzing symptom progression through sequential or time-series models would enable the prediction not only of presence but also of diabetes stage or severity.

Explainability and Trust: The use of explainable AI (XAI) can explain how individual symptoms (such as polyuria, alopecia) contribute to the ultimate prediction and establish trust between clinicians and patients.

Thank You